

MODULE 1

Microservices Fundamentals

From monoliths to distributed services — the principles, tradeoffs and practices that make them work.

**From monoliths to
distributed services**

**Microservices
principles**

**Benefits and
tradeoffs**

Twelve-Factor App

**Microservices +
DevOps**

LEARNING OBJECTIVES

What you will understand by the end of this module.



BY THE END OF THIS MODULE, YOU CAN

1. Explain monolithic architecture limitations
2. Describe microservices principles
3. Identify benefits and tradeoffs
4. Choose between monoliths and microservices
5. Explain Twelve-Factor App principles
6. Connect microservices with DevOps



MODULE ROADMAP



SIX QUESTIONS THIS MODULE ANSWERS

WHY

did microservices become popular?

WHAT

exactly is a microservice?

HOW

should services be separated?

WHAT PROBLEMS

do microservices solve?

WHAT NEW PROBLEMS

do they introduce?

HOW

do DevOps, containers and OpenShift support them?

KEY TAKEAWAY Choosing microservices is an architecture decision — not automatically a modernization decision.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 **Monolith**

One deployable application where every feature shares one codebase and usually one database.

2 **Microservice**

A small, independently deployable service built around one business capability.

3 **Business capability**

Something the business does, such as taking payments, that one service owns end to end.

4 **Independent deployment**

Releasing one service without redeploying or coordinating the others.

5 **Fault isolation**

Designing so a failure in one service doesn't take the rest of the system down.

6 **Decentralized data ownership**

Each service owns its data; others reach it only through that service's API or events.

7 **Twelve-Factor App**

Twelve practices for cloud-ready apps, such as config in the environment and stateless processes.

8 **Modular monolith**

One deployable app split into strict internal modules — often a step before microservices.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

API

Application Programming Interface

ESB

Enterprise Service Bus

SRE

Site Reliability Engineering

CI

Continuous Integration

HTTP

HyperText Transfer Protocol

UI

User Interface

CI/CD

Continuous Integration /
Continuous Delivery

MVP

Minimum Viable Product

URL

Uniform Resource Locator

DB

Database

REST

Representational State Transfer

DDD

Domain-Driven Design

SOA

Service-Oriented Architecture

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

ONE STORY THROUGH THE MODULE

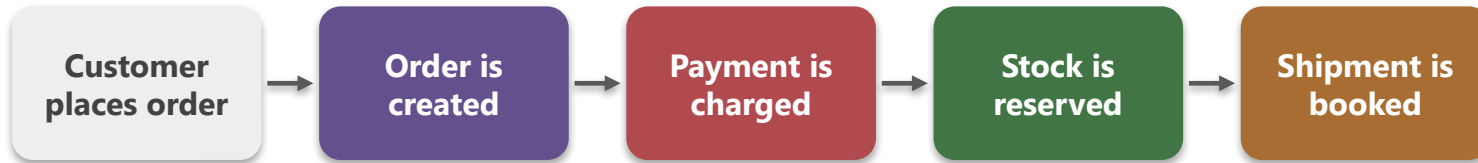
An e-commerce order platform — and the banking lab you will build.



THE E-COMMERCE ORDER PLATFORM



ONE CHECKOUT, STEP BY STEP



THE MODULE'S QUESTION Should all of this live in one application — or in separate services? And what does each choice cost?

YOUR LAB: BANKING

Lab 1 builds an Account Service for the course's banking platform. Every idea you see on the order platform applies there too.

LAB SCOPE

Account

Transaction

Risk Assessment

KEY TAKEAWAY The same order platform runs through every slide, so each new idea builds on the last.

PART 1: WHY MONOLITHS STRUGGLE

Start with the architecture most systems begin with.



SO FAR

- ✓ You know the goals of this module
- ✓ You met the e-commerce order platform

NEXT QUESTION

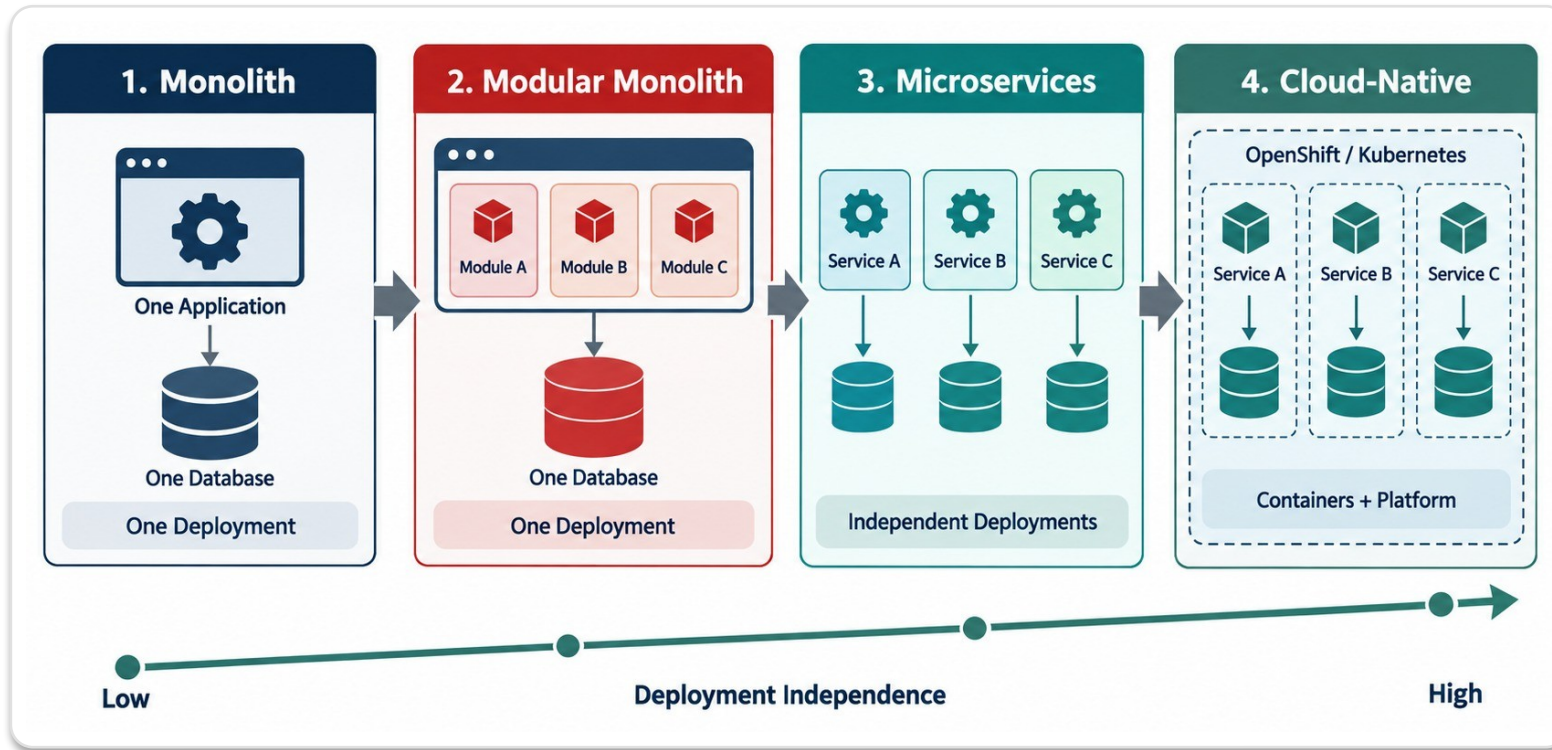
Why do successful monolithic applications become hard to change and scale?

IN THIS PART

- 1 How application architecture evolved
- 2 SOA versus microservices
- 3 What a monolith really is
- 4 Where monoliths start to hurt
- 5 The scaling problem

EVOLUTION OF APPLICATION ARCHITECTURE

Monolith → modular monolith → microservices → cloud-native.



Delivery evolved too

Manual deploys became CI/CD, servers became containers, static infrastructure became OpenShift.

NOT A LADDER A well-designed monolith can beat badly designed microservices.

KEY TAKEAWAY Architecture evolves as systems, teams and delivery needs grow — not simply because a newer style exists.

SOA VS. MICROSERVICES

Both reuse services — but they differ in scope, integration and ownership.

Aspect	Service-Oriented Architecture	Microservices
Scope	Enterprise-wide integration	One business capability
Integration	Central ESB routes and transforms	Lightweight REST, gRPC, events
Data	Often shared enterprise databases	Each service owns its data
Governance	Central standards and control	Team autonomy with guardrails
Deployment	Services often released together	Independently deployable
Change	Coordinated and slower	Isolated and faster

What they share

Both expose capabilities through service interfaces for reuse.

The key difference

SOA centralizes integration; microservices decentralize ownership.

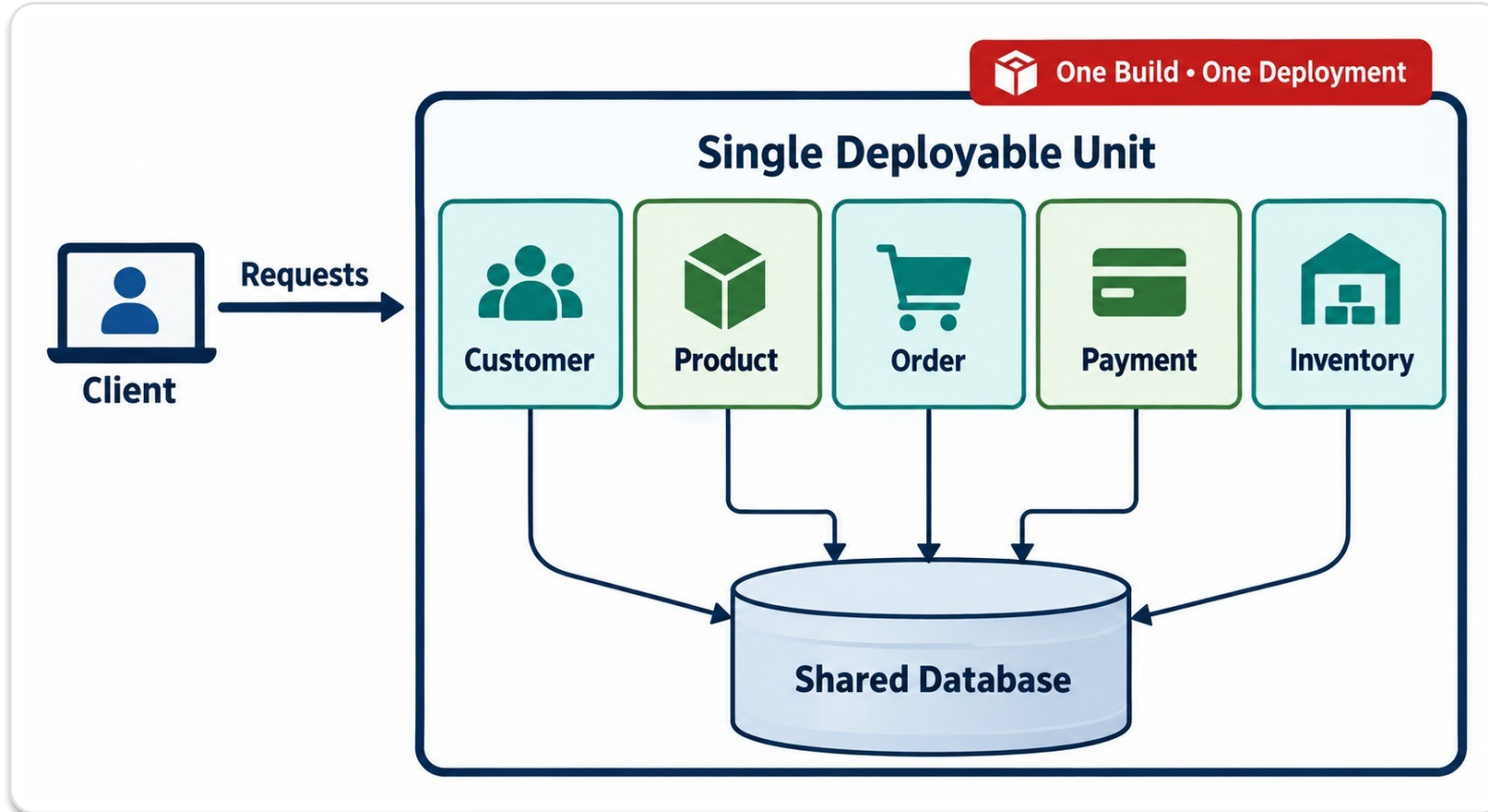
WHY IT MATTERS A central ESB can become the new monolith.

KEY TAKEAWAY SOA focused on enterprise integration and reuse; microservices focus on independent delivery of business capabilities.



UNDERSTANDING MONOLITHIC ARCHITECTURE

One application, one deployment unit, one shared database.



One artifact

```
CustomerController  OrderController  
PaymentController   InventoryController  
→ commerce.jar
```

One unit

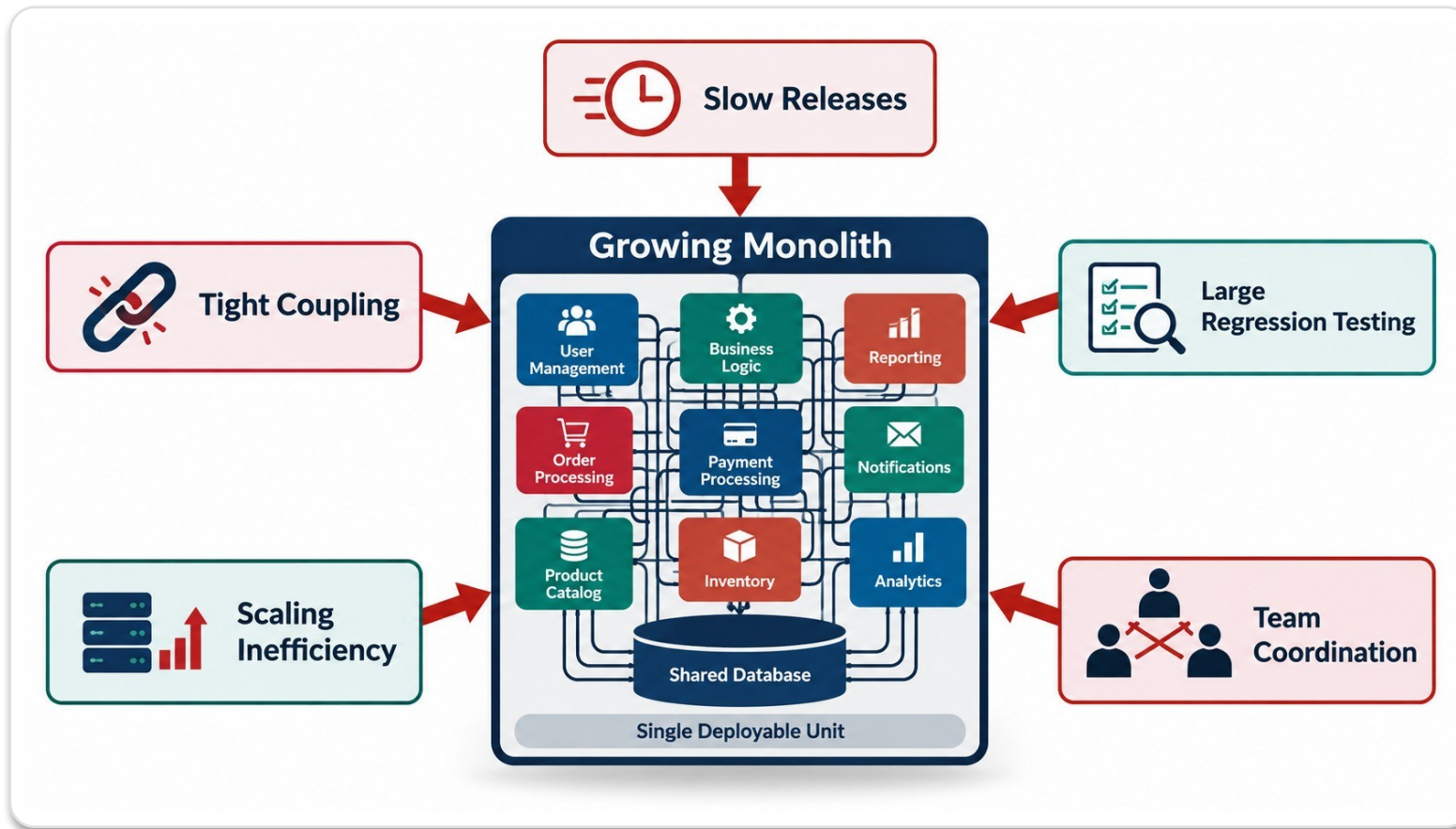
Everything builds, tests and deploys together and shares one database.

NOT BAD DESIGN A clean, modular monolith is often the right start.

KEY TAKEAWAY One application, one deployment unit — and monolithic does not automatically mean bad design.

PROBLEMS WITH MONOLITHIC ARCHITECTURES

Coupling, deployment risk, scaling and a growing codebase.



As it grows

More features, a larger codebase, more dependencies — and slower, riskier releases.

100 developers, one app

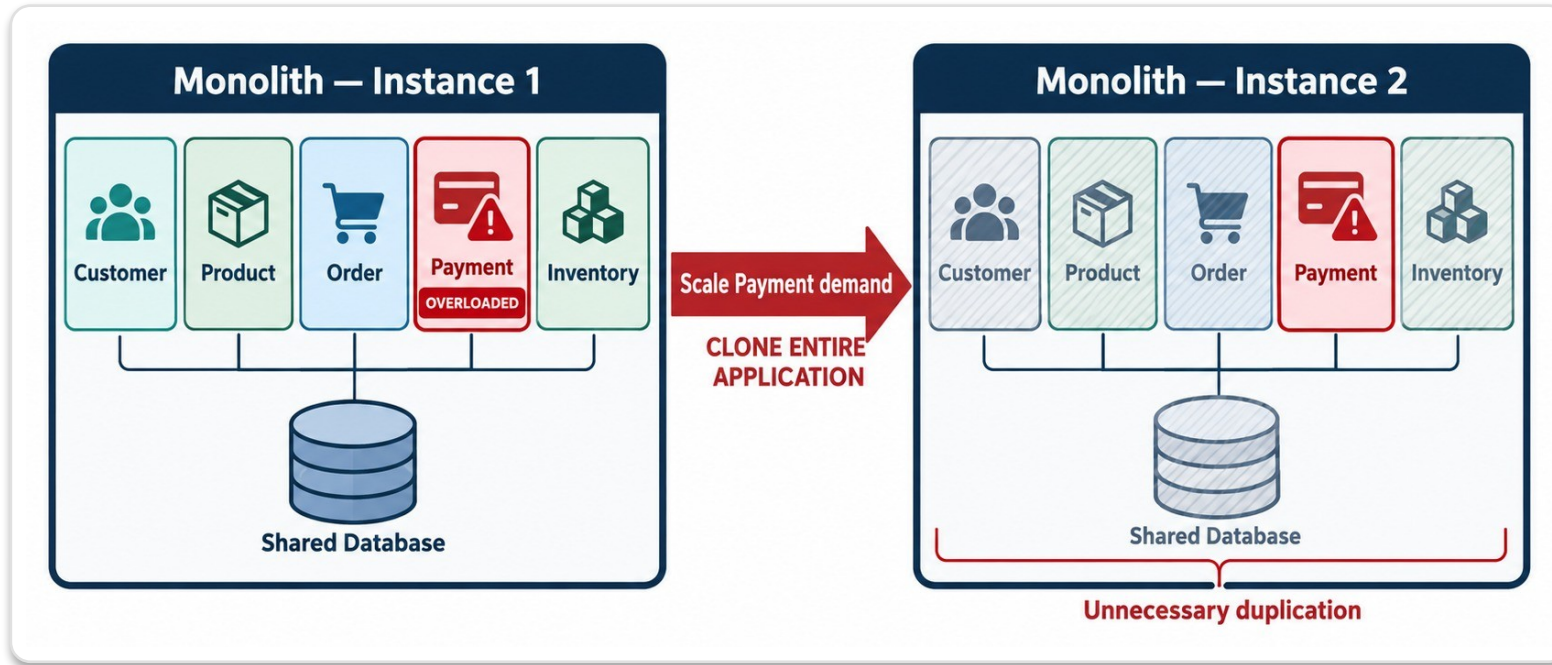
A small payment change still means build all, test all, deploy all.

DEPLOYMENT COUPLING Every change waits for one shared release.

KEY TAKEAWAY A monolith is not automatically tightly coupled inside — good modular design greatly reduces these problems.

SCALING CHALLENGES IN A MONOLITH

Payment is overloaded, so the whole application is copied.



Load by capability

Payment runs at 95% while Customer, Product and Inventory sit at 20–40%.

Monolith

Adding Payment capacity means copying every capability.

Microservices

Run ten Payment instances and two Customer instances.

FAIR POINT Monoliths scale horizontally too — just not per capability.

KEY TAKEAWAY Monoliths can scale horizontally too — the microservices advantage is fine-grained scaling by service.

PART 2: WHAT MICROSERVICES ARE

From the pain of one big application to independent services.



SO FAR

- ✓ Monoliths are simple to start with
- ✓ As they grow, releases, teams and scaling become coupled
- ✓ Scaling one hot capability means copying everything

NEXT QUESTION

What changes when we split capabilities into independently deployable services?

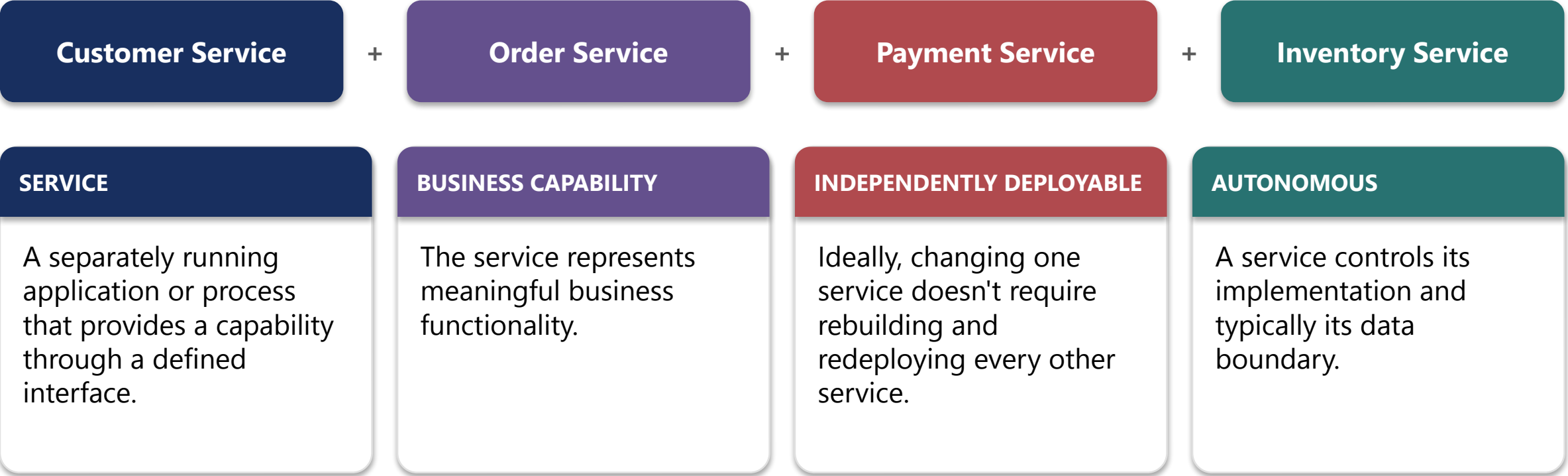
IN THIS PART

- 1 The definition of a microservice
- 2 The architecture at a glance
- 3 Local calls become network calls
- 4 The core principles
- 5 Capabilities, deployment and data ownership
- 6 Synchronous and asynchronous communication

WHAT ARE MICROSERVICES?

The definition and the fundamental idea.

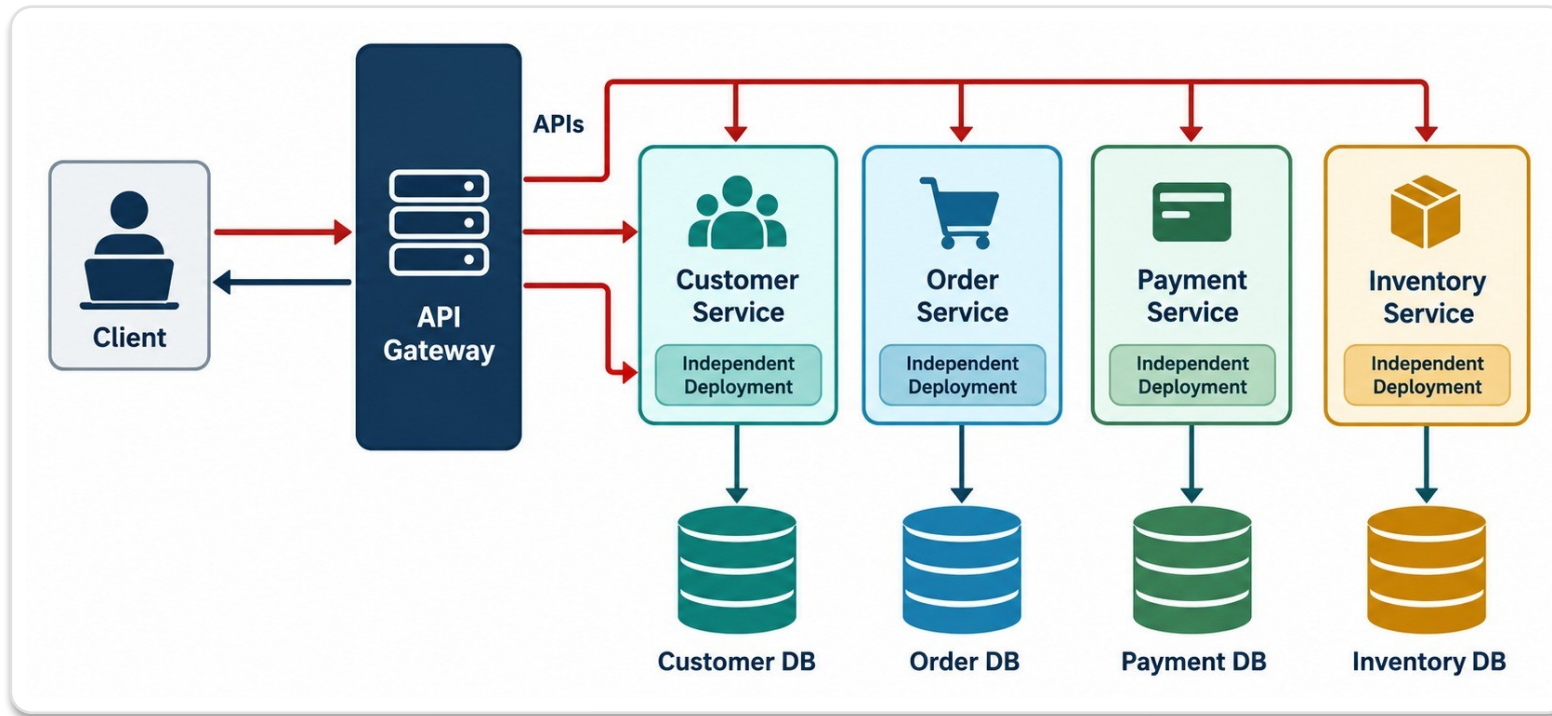
“A system composed of independently deployable services aligned with business capabilities.”



KEY TAKEAWAY A Payment Service owns payment behavior — it is not a generic collection of unrelated functions.

MICROSERVICES ARCHITECTURE OVERVIEW

Clients reach independent services through an API gateway.



API gateway

One entry point that routes each request to the right service.

Independent services

Customer, Order, Payment and Inventory run and deploy on their own.

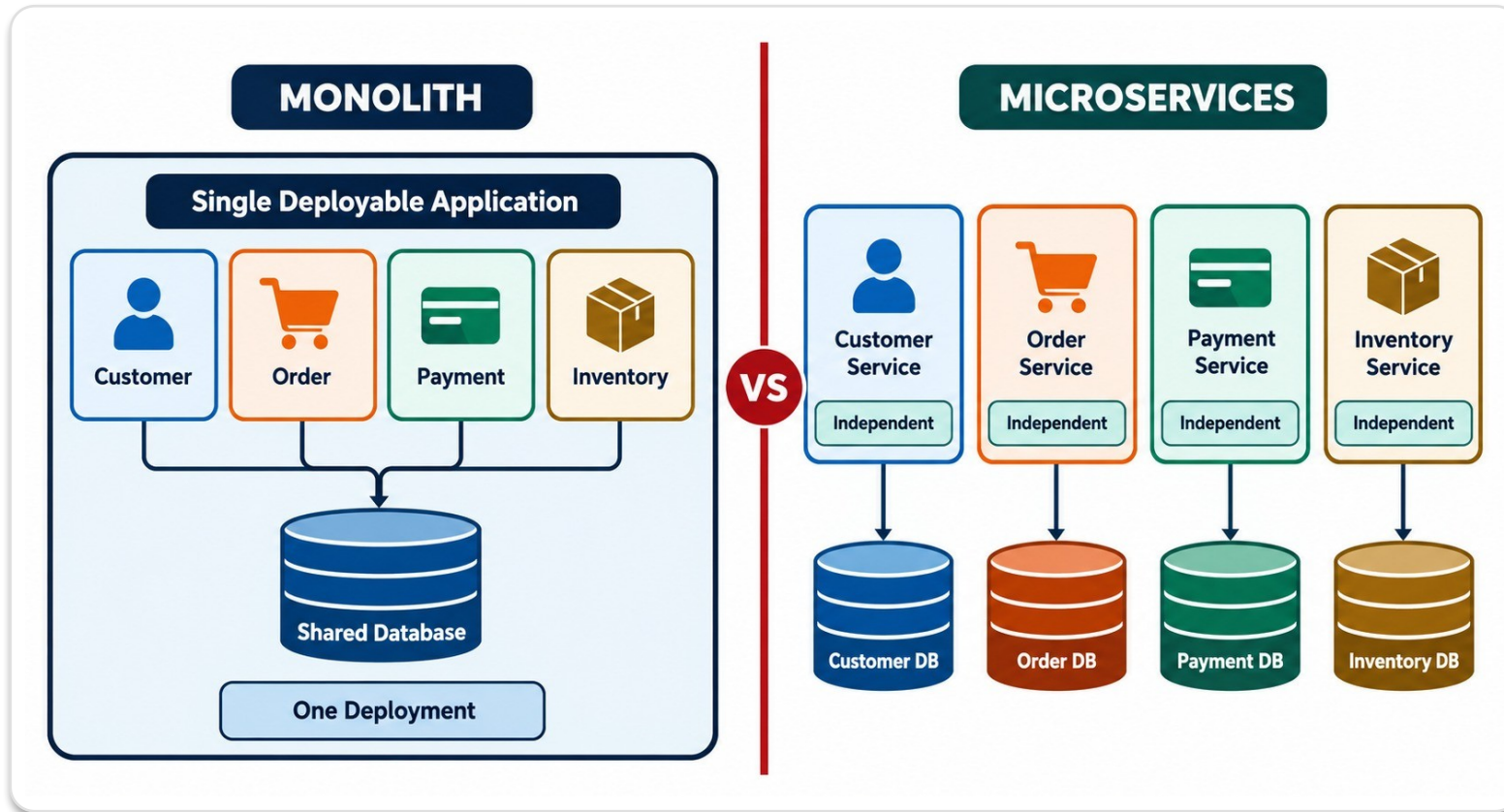
Owned data

Each service keeps its own database — no shared tables.

KEY TAKEAWAY Clients reach independent services through a gateway, and each service owns its data.

MONOLITH VS. MICROSERVICES ARCHITECTURE

The same capabilities — in one process, or across the network.



Monolith: local call

```
paymentService.charge(order);
```

Microservices: remote call

```
paymentClient.charge(order); // HTTP
```

THE NETWORK ADDS

Latency

Timeouts

Partial failure

Security

API contracts

Versioning

KEY TAKEAWAY Crossing the network boundary changes everything: latency, security, timeouts and partial failures.

FROM A LOCAL CALL TO A REMOTE CALL

The same checkout step, before and after the split.

Monolith — one transaction

```
@Transactional
public Order placeOrder(Cart cart) {
    Order order = orders.save(Order.from(cart));
    inventory.reserve(cart.items()); // local
    payments.charge(order);          // local
    return order;                   // one commit
}
```

Microservices — calls cross the network

```
public Order placeOrder(Cart cart) {
    Order order = orders.save(Order.pending(cart));
    inventoryClient.reserve(order.id(), cart.items());
    paymentClient.charge(order.id(), order.total());
    return order; // no single transaction now
}
```

What you lose

One database transaction no longer covers order, stock and payment.

What you gain

Payment and Inventory deploy and scale on their own.

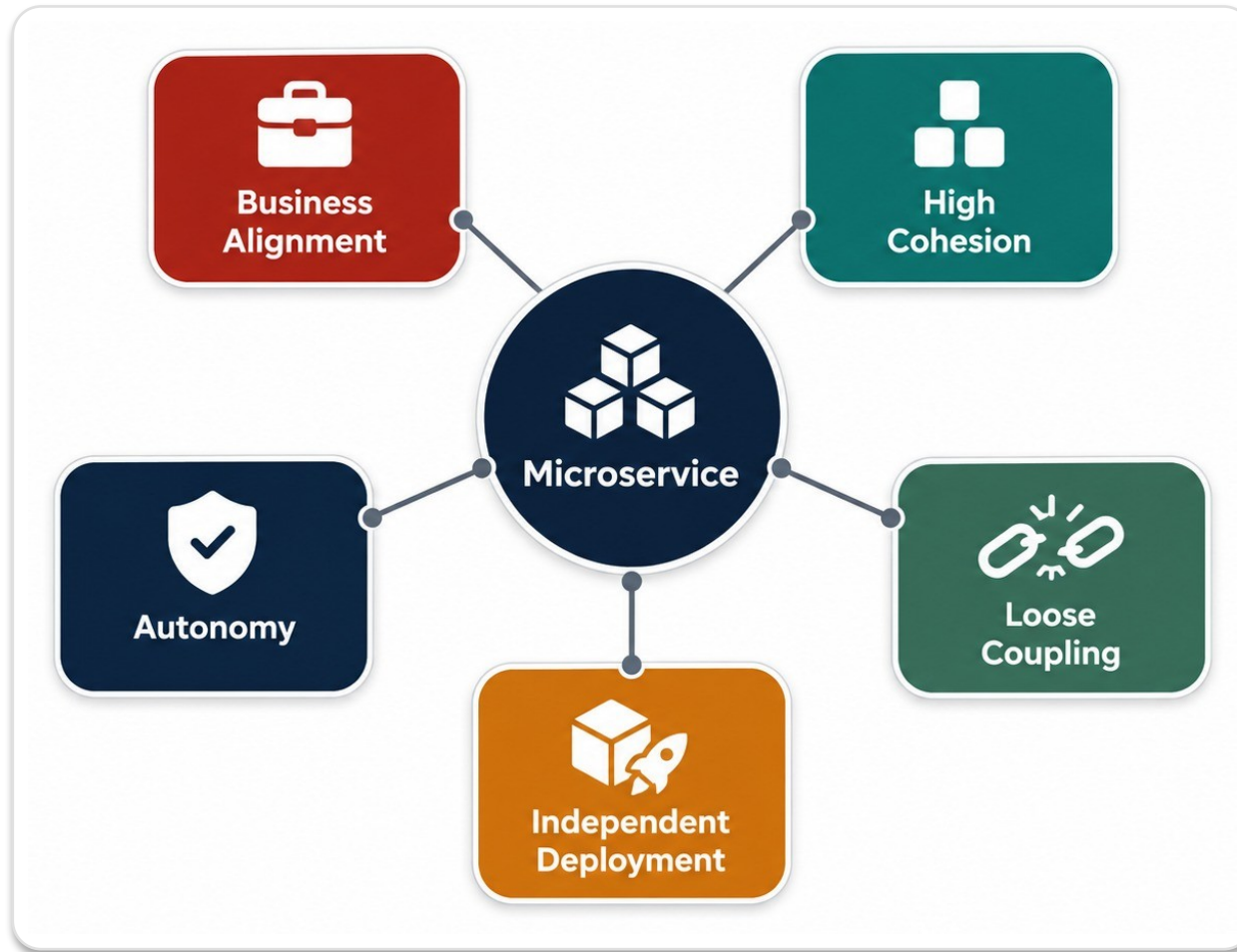
What you need

Timeouts, retries and a plan for partial failure — Part 3.

KEY TAKEAWAY A method call becomes a network call — great for independence, but it changes consistency and failure handling.

CORE PRINCIPLES OF MICROSERVICES

Five principles that matter more than making services small.



EACH IN ONE LINE

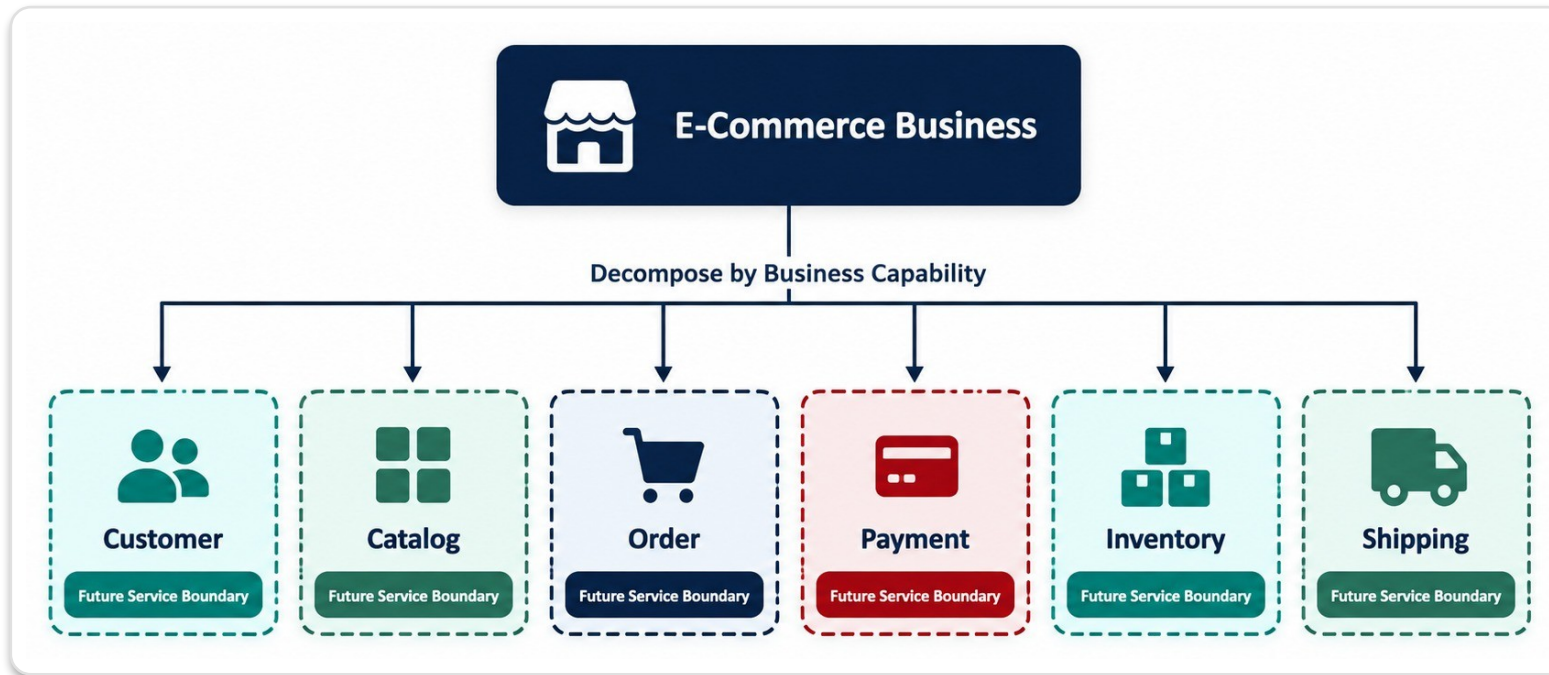
- ✓ Business alignment: follow business domains
- ✓ High cohesion: related behavior stays together
- ✓ Loose coupling: know contracts, not internals
- ✓ Autonomy: a service controls its own behavior
- ✓ Independent deployment: release without coordination

NOT ABOUT SIZE A small service can still break every principle.

KEY TAKEAWAY These principles are more important than simply making services small.

DESIGNING AROUND BUSINESS CAPABILITIES

Cut services by what the business does, not by technical layer.



AVOID TECHNICAL-LAYER SERVICES

- ✗ UI service
- ✗ Business-logic service
- ✗ Database service

Order Service owns

Create, update and cancel orders, and order status.

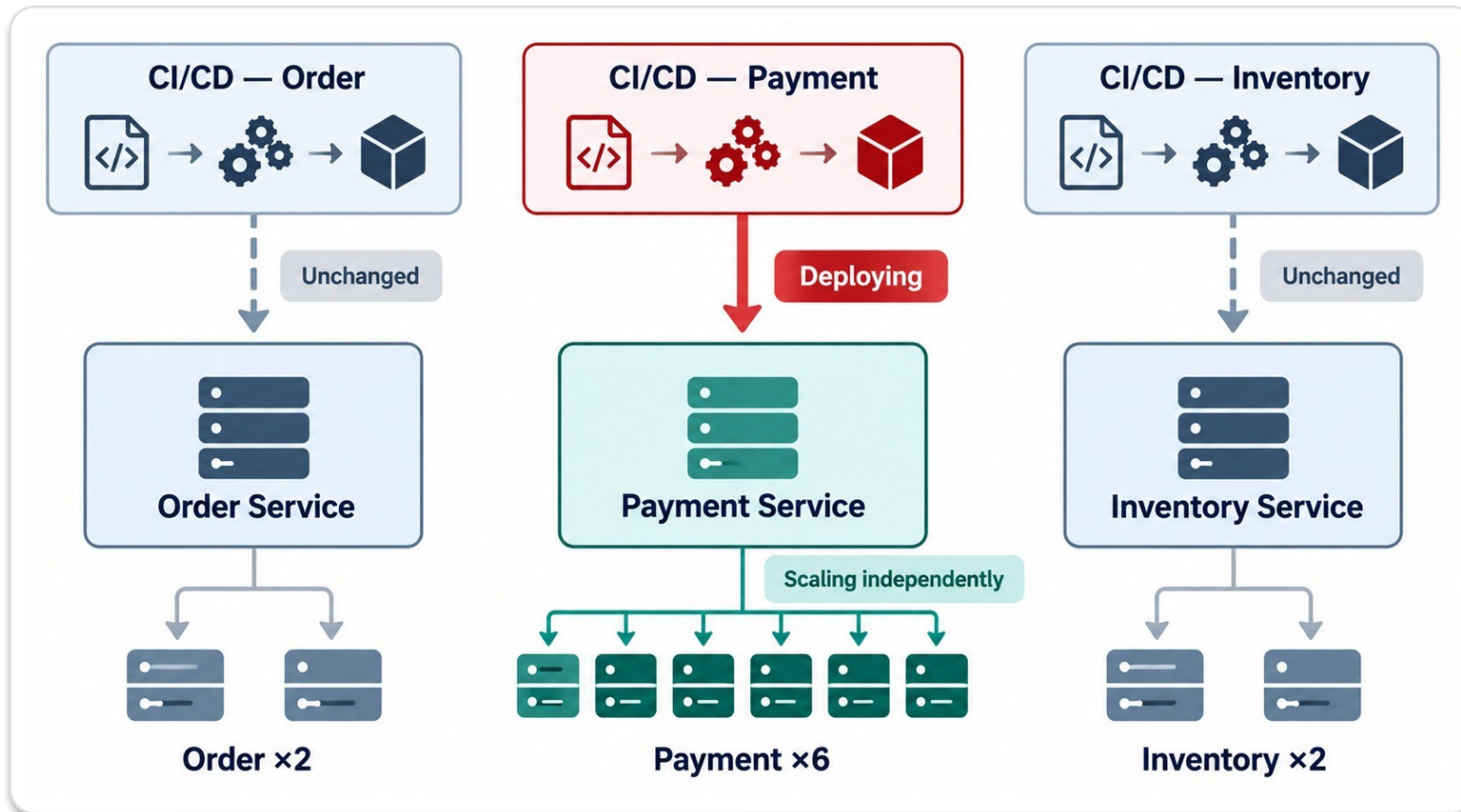
Payment Service owns

Authorize, capture and refund payments.

KEY TAKEAWAY Draw boundaries around business domains, not technical layers — later this leads to DDD and bounded contexts.

INDEPENDENT DEPLOYMENT AND SCALING

Deploy and scale one service without redeploying the others.



One pipeline per service

Git commit → CI → tests → image → registry → OpenShift.

Only Payment changes

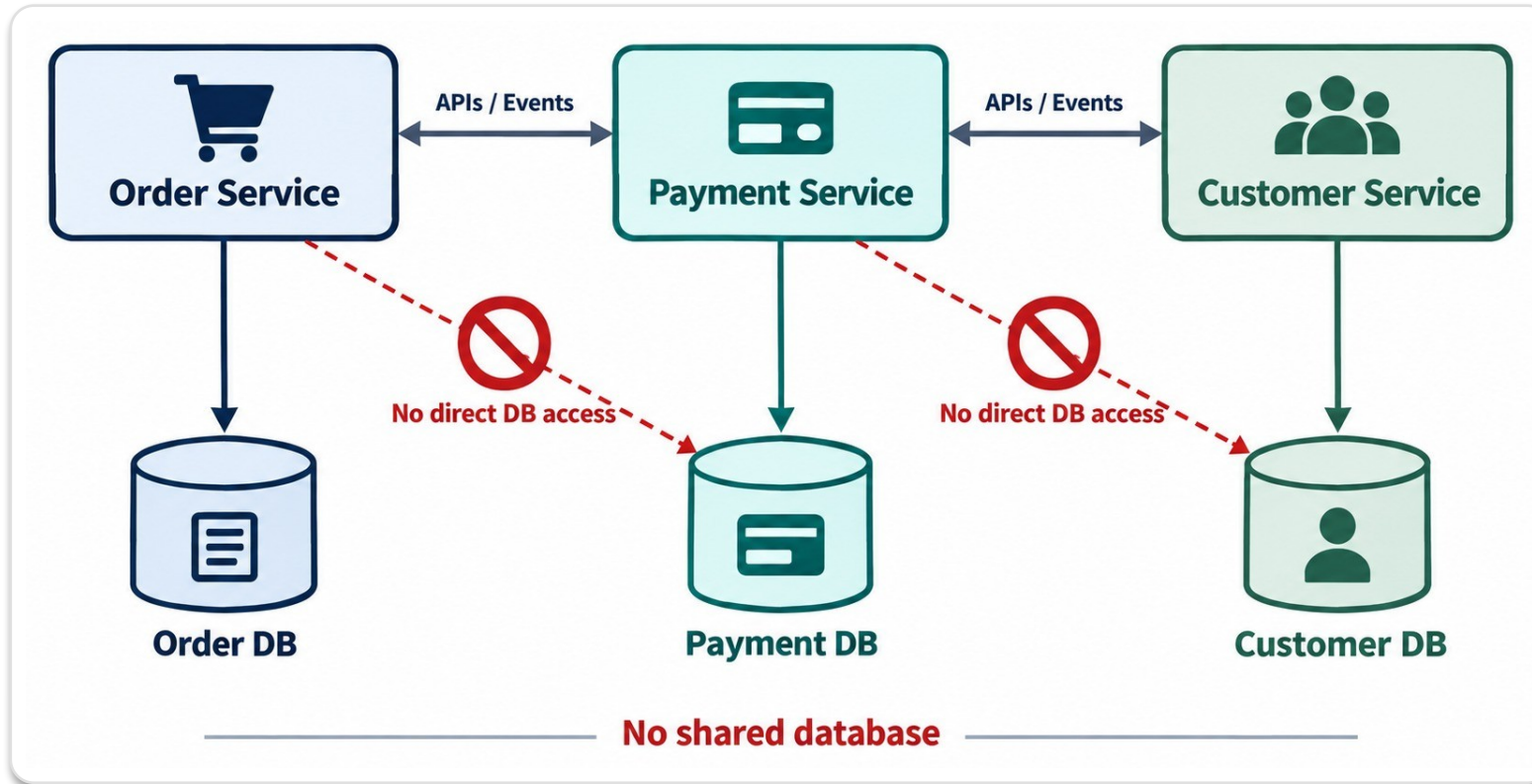
Payment ships a fix while Customer, Order and Inventory keep running.

NEEDS AUTOMATION Independence works only with automated tests and deployment.

KEY TAKEAWAY Change Payment Service, then build, test, deploy and scale it — without redeploying unrelated services.

DECENTRALIZED DATA OWNERSHIP

Database-per-service and clear data boundaries.



AVOID

- ✗ One shared database for every service
- ✗ Reading or writing another service's tables

PREFER

- ✓ Each service owns its data
- ✓ Share data through APIs or events

THE BIG CHALLENGE Keeping data consistent across services — covered in the data module.

KEY TAKEAWAY Each service owns its data — if services share tables, the database becomes a coupling mechanism.

OWNING DATA IN CODE

Ask the owner — through its API or an event — instead of touching its database.

✗ Writes another service's table

```
// Order Service reaching into  
// Inventory's database  
inventoryRepository  
    .decrementStock(sku, qty);
```

✓ Calls the owner's API

```
inventoryClient.reserve(  
    new Reservation(orderId,  
                    sku, qty));
```

✓ Publishes an event

```
events.publish(new OrderPlaced(  
    orderId, customerId,  
    items, total));
```

Why

Inventory keeps control of its own rules, schema and changes.

API or event?

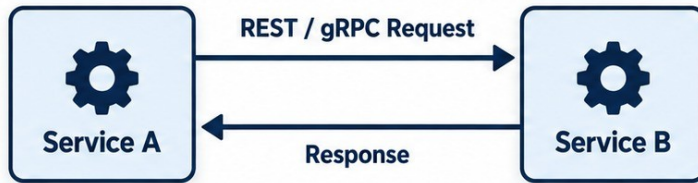
Need an answer now → call the API. Others react later → publish an event.

KEY TAKEAWAY Services change each other's data only by asking the owner — never by touching its tables.

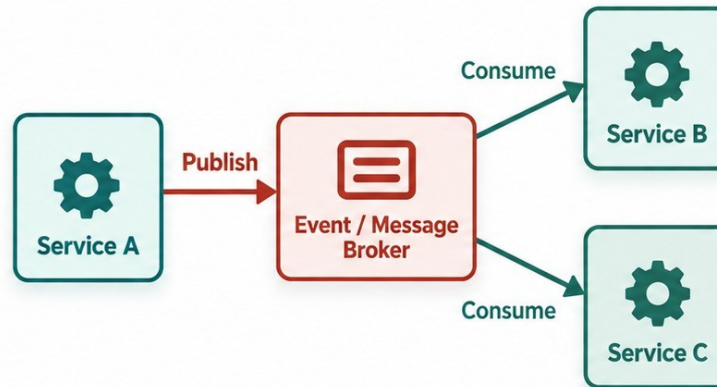
MICROSERVICE COMMUNICATION PATTERNS

REST, gRPC, messaging and events.

Synchronous



Asynchronous



COMMON OPTIONS

REST

gRPC

Events

Messaging

Synchronous

The caller sends a request and waits for the reply.

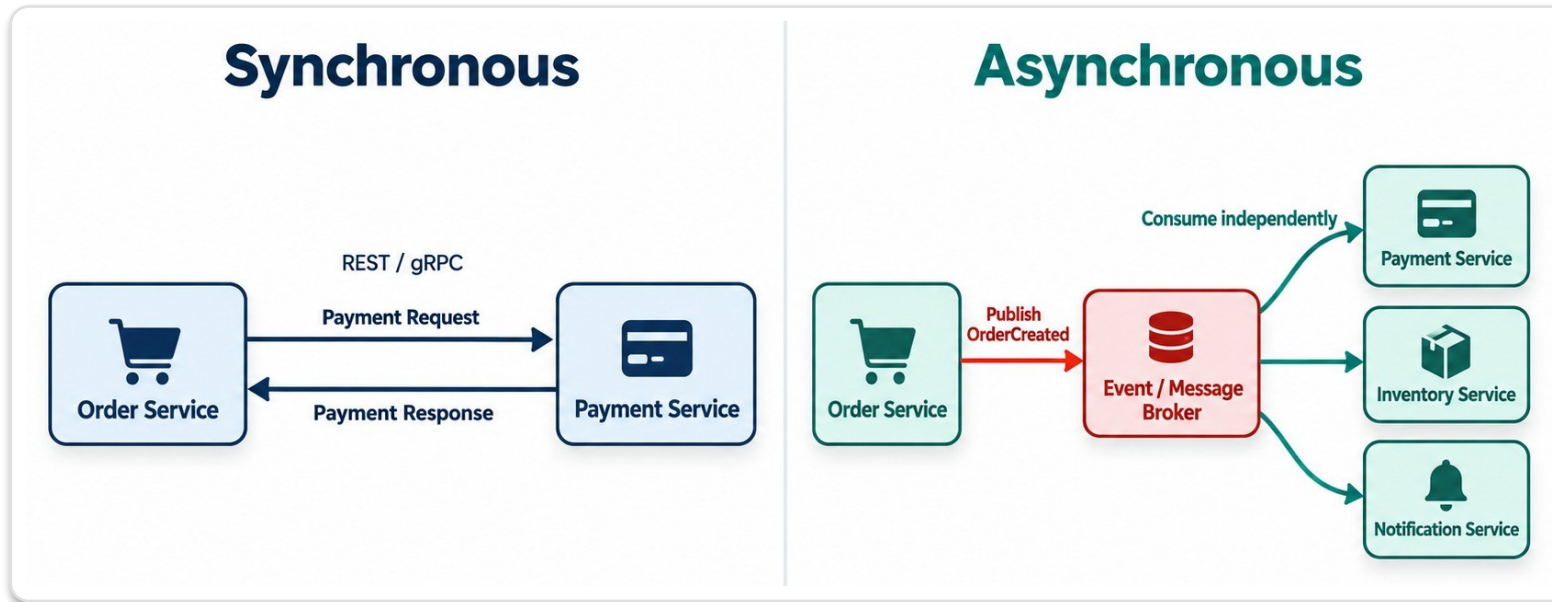
Asynchronous

The producer publishes and moves on; consumers react later.

KEY TAKEAWAY Services must communicate — choose request/response or events based on what each interaction needs.

SYNCHRONOUS VS. ASYNCHRONOUS COMMUNICATION

The same order flow — once as a request, once as events.



Checkout: synchronous

Order asks Payment and waits for the result.

After acceptance: events

OrderCreated lets Inventory, Notification and Analytics react on their own.

USE BOTH Most systems combine request/response and events.

KEY TAKEAWAY Neither approach is universally better — most enterprise microservice architectures use a combination.

KEY CHARACTERISTICS OF MICROSERVICES

Small, autonomous, independently deployable — and observable.

✓ A WELL-DESIGNED MICROSERVICE IS



Business focused



Loosely coupled



Highly cohesive



Independently deployable



Independently scalable



Observable



Resilient

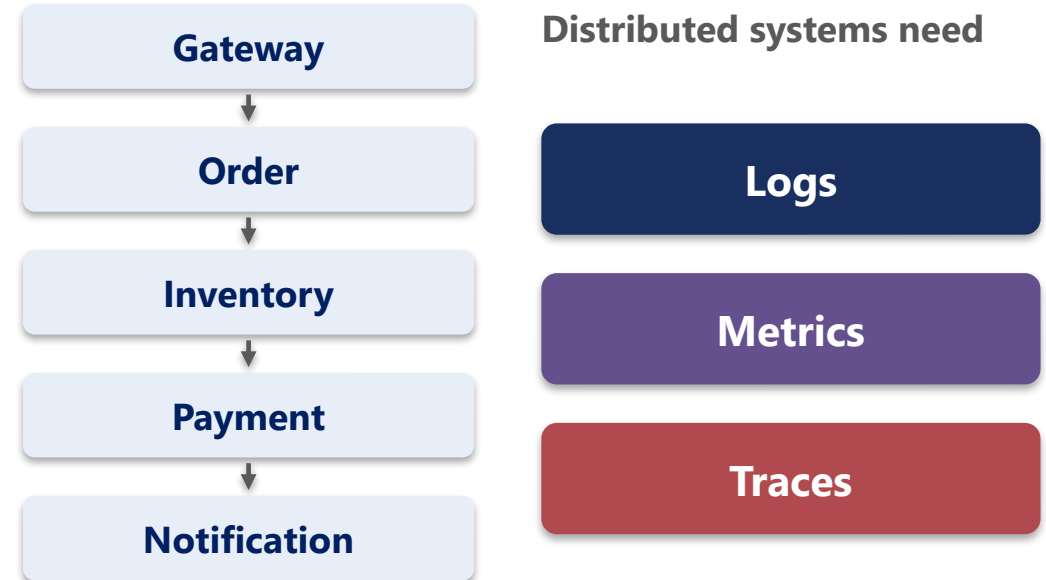


Automation friendly



WHY OBSERVABLE MATTERS

In a monolith you might read one application's logs. With 50 services, one request may travel through:



KEY TAKEAWAY Finding where a request failed across many services is hard — distributed systems need logs, metrics and traces.

PART 3: COSTS AND DECISIONS

Independence has a price. Decide when it's worth paying.



SO FAR

- ✓ A microservice owns one business capability
- ✓ Services deploy, scale and store data independently
- ✓ They talk through APIs and events

NEXT QUESTION

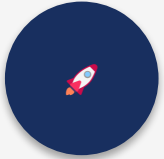
What does this independence cost — and when is it worth paying?

IN THIS PART

- 1 The benefits you can get
- 2 Distributed-system challenges
- 3 Designing for failure, in code
- 4 Misconceptions and anti-patterns
- 5 Monolith or microservices?
- 6 The modular monolith

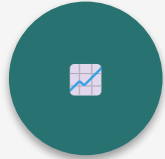
BENEFITS OF MICROSERVICES

Agility, scalability, autonomy and fault isolation.



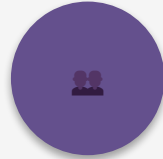
Independent Delivery

Teams can release services at different cadences.



Independent Scaling

High-demand services can receive more resources.



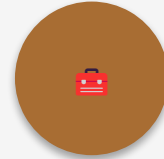
Team Autonomy

Different teams can own specific business capabilities.



Fault Isolation

Properly designed, a failure may stay confined to one service.



Technology Flexibility

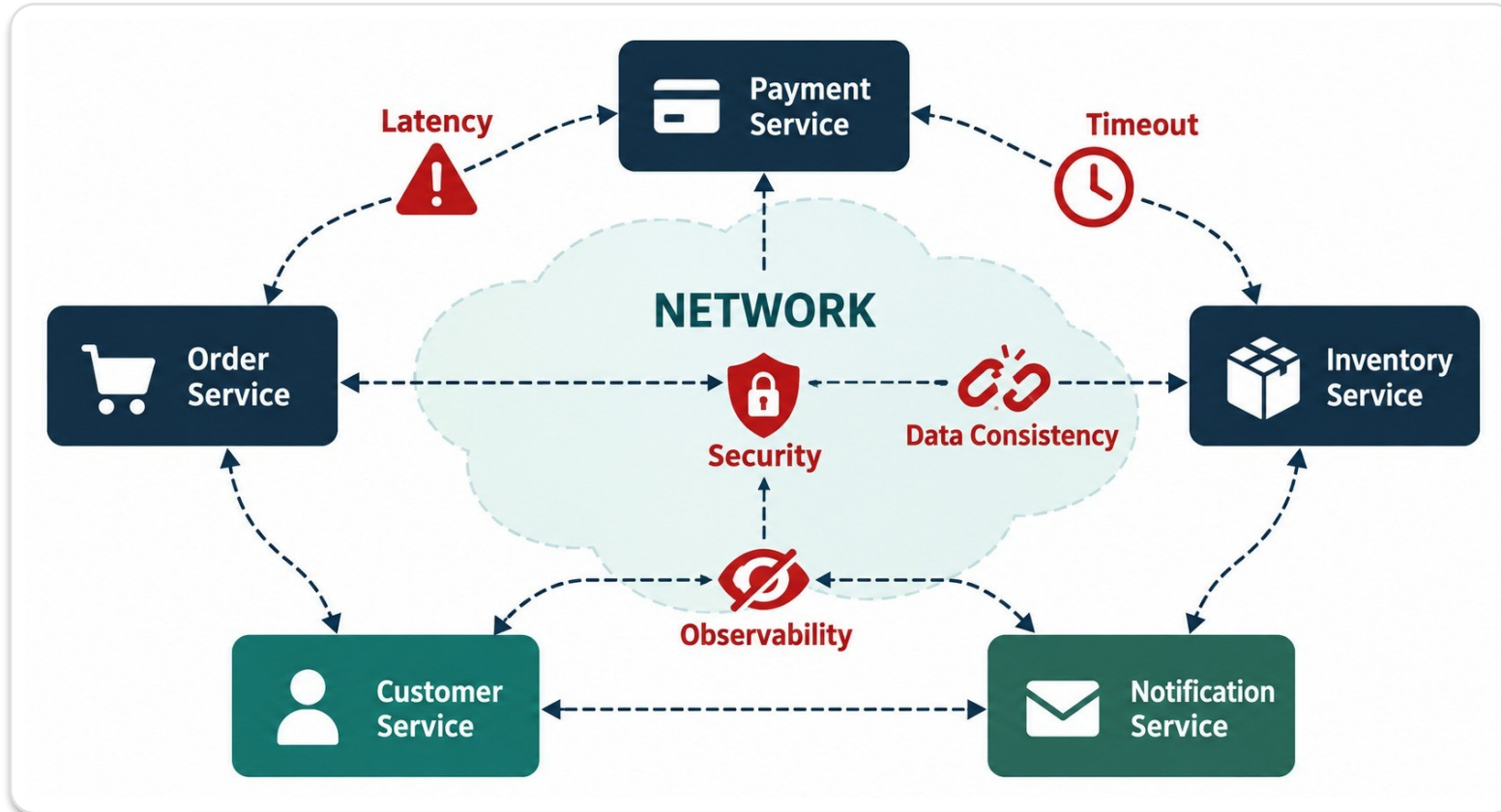
Different services can potentially use different technologies.

POTENTIAL, NOT GUARANTEED Poor service boundaries can produce a distributed system that is harder to maintain than the original monolith.

KEY TAKEAWAY These are potential benefits, not automatic guarantees — they depend on good boundaries and engineering.

MICROSERVICES TRADEOFFS AND CHALLENGES

Network, consistency, observability, security and operations.



A REMOTE SERVICE COULD

- × Be unavailable or slow
- × Return an error
- × Process the request but lose the reply
- × Run a different API version

YOU INHERIT

Network failures

Data consistency

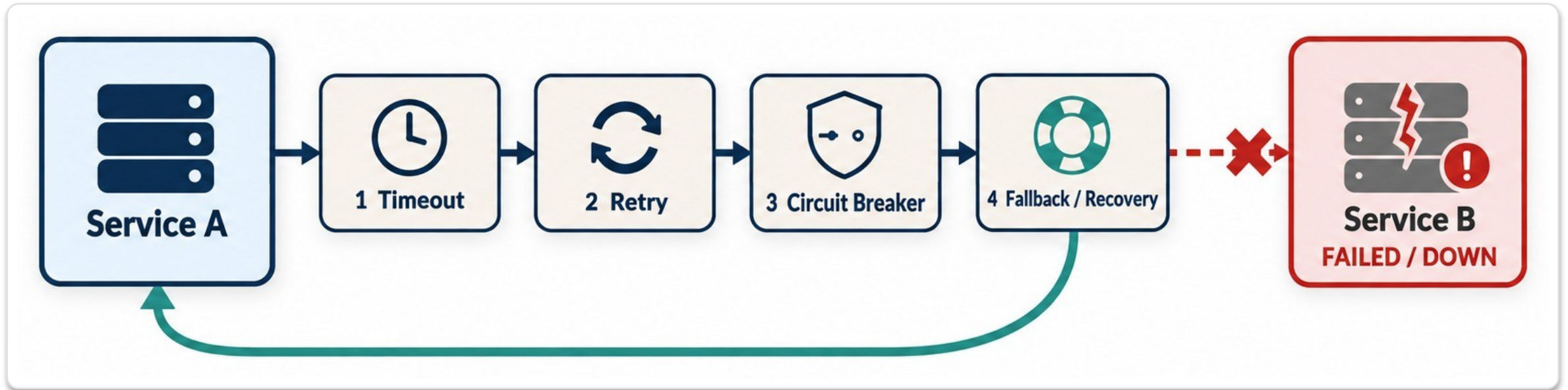
Security

Observability

KEY TAKEAWAY Microservices therefore require more mature engineering practices.

DESIGNING FOR FAILURE

Timeouts, retries, circuit breakers, fallbacks and health checks.



Timeout

Don't wait forever for a slow service.

Retry

Retry only safe, temporary failures — with limits.

Circuit breaker

Stop calling a failing dependency for a while.

Fallback

Return a safe alternative, e.g. mark the order pending.

Health checks

Tell the platform whether an instance is healthy.

KEY TAKEAWAY Timeouts, controlled retries, circuit breakers, fallbacks and health checks stop one failure from spreading.

DESIGNING FOR FAILURE IN CODE

Checkout keeps working when Payment Service is slow or down.

1 Put a timeout on the client

```
var factory = new SimpleClientHttpRequestFactory();
factory.setConnectTimeout(Duration.ofSeconds(1));
factory.setReadTimeout(Duration.ofSeconds(2));

RestClient payments = RestClient.builder()
    .baseUrl("http://payment-service")
    .requestFactory(factory)
    .build();
```

Timeout

A slow Payment Service can't hold Order's threads forever.

✓ 2 Fall back instead of failing the order

```
public OrderResult checkout(Order order) {
    try {
        payments.post().uri("/payments")
            .body(PaymentRequest.of(order))
            .retrieve().toBodilessEntity();
        return OrderResult.confirmed(order.id());
    } catch (RestClientException ex) {
        log.warn("Payment unavailable for {}", order.id());
        return OrderResult.pendingPayment(order.id());
    }
}
```

Fallback

The order is kept as PENDING_PAYMENT instead of failing.

LATER IN THE COURSE Resilience4j adds retries and circuit breakers.

KEY TAKEAWAY Bound every remote call with a timeout, and decide what the business does when it fails.

EXERCISE: CHECKOUT WHEN PAYMENT IS DOWN

Decide how the order platform should behave during a partial failure.

PRACTICE EXERCISE

10 MIN

SCENARIO

A customer checks out a \$180 order. Order Service calls Payment Service, but Payment has been timing out for three minutes.

YOUR TASKS

- 1 Choose: reject, retry, keep pending, or queue for later
- 2 Say what the customer sees
- 3 Decide what happens to the reserved stock
- 4 Name the signal that alerts the team

EXPECTED OUTCOME

- ✓ Don't blindly retry the charge — it could double-charge
- ✓ Keep the order as PENDING_PAYMENT and tell the customer
- ✓ Retry in the background; release stock after a time limit
- ✓ Alert on payment error rate and latency

KEY TAKEAWAY Failure handling is a business decision, not only a technical one.

MISCONCEPTIONS AND ANTI-PATTERNS

Common beliefs that lead teams into trouble.

MISCONCEPTION

REALITY

ANTI-PATTERNS TO AVOID

Microservices are always faster

Poorly designed ones are slower than a good monolith

They're just small services

They're defined by business boundaries, not size

Splitting is just moving code

Data, teams, testing and operations change too

Every service needs new tech

Too much variety makes support harder

Microservices = cloud + DevOps

They're an architecture; cloud and DevOps enable it

- ✗ Too many tiny services
- ✗ One shared database
- ✗ Synchronous, chatty calls everywhere
- ✗ Services that must deploy together
- ✗ Big-bang rewrite
- ✗ No logs, metrics or traces

REMEMBER Microservices amplify good design — and bad design.

KEY TAKEAWAY Microservices amplify both good and bad design — avoid these traps from the start.

WHEN TO USE MICROSERVICES VS. A MONOLITH

Practical architecture decision criteria.

CONSIDER MICROSERVICES WHEN

- ✓ Multiple autonomous teams
- ✓ Clear domain boundaries
- ✓ Independent releases required
- ✓ Different scaling requirements
- ✓ Mature CI/CD and operations

CONSIDER A MONOLITH WHEN

- ✓ Small team
- ✓ New / MVP application
- ✓ Uncertain domain
- ✓ Simple deployment
- ✓ Limited operational complexity desired

ASK FIRST — *don't start with "Microservices are modern, so we should use microservices."*

1

Do teams need independent deployment?

2

Are domain boundaries sufficiently understood?

3

Do capabilities have very different scaling needs?

4

Can the organization operate distributed systems effectively?

5

Does the added complexity solve a meaningful problem?

KEY TAKEAWAY Start from business and engineering requirements — if the answers are no, a monolith may be the better choice.

EXERCISE: MONOLITH OR MICROSERVICES?

Apply the decision criteria to four situations.

PRACTICE EXERCISE

10 MIN

SCENARIO

A: an internal tool for five staff to approve supply requests.
B: a checkout platform with seasonal spikes and six product teams.
C: a brand-new subscription product whose rules change weekly.
D: fraud scoring that changes daily and needs its own scaling.

YOUR TASKS

- 1 Pick monolith, modular monolith or microservices for each
- 2 Justify it with team size, domain clarity and scaling needs
- 3 Name one risk of your choice

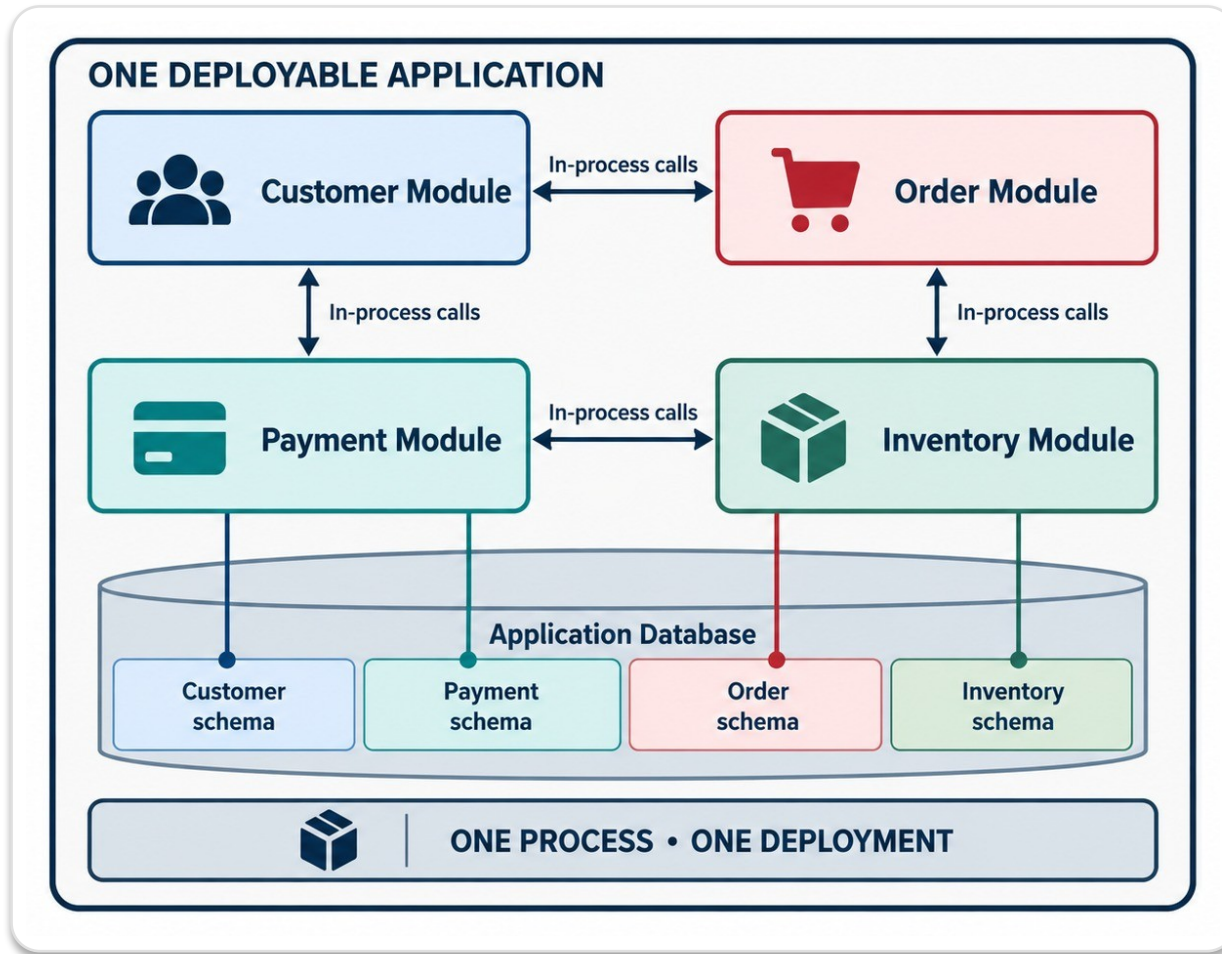
EXPECTED DIRECTION

- ✓ A → Monolith
- ✓ B → Microservices
- ✓ C → Modular monolith first
- ✓ D → A separate microservice

KEY TAKEAWAY The skill is explaining why — not just picking a label.

THE MODULAR MONOLITH ALTERNATIVE

Strong module boundaries inside one deployable application.



Start here

Customer, Order, Payment and Inventory as strong modules in one application.

Evolve when needed

Extract a module only when it truly needs its own deployment or scaling.

AVOID PREMATURE DISTRIBUTION Keep one deployment until distribution solves a real problem.

KEY TAKEAWAY A modular monolith is an excellent option — microservices should not always be the starting point.

PART 4: CLOUD-NATIVE AND DEVOPS

From architecture choices to services that run well on OpenShift.



SO FAR

- ✓ Microservices trade simplicity for independence
- ✓ Design for failure from day one
- ✓ Choose microservices only when the need is real

NEXT QUESTION

How do we build services that run well in containers and on OpenShift?

IN THIS PART

- 1 The Twelve-Factor App
- 2 Twelve-Factor in Spring Boot code
- 3 Twelve-Factor on OpenShift
- 4 Conway's Law and team ownership
- 5 Microservices in DevOps teams
- 6 The end-to-end delivery path

INTRODUCTION TO THE TWELVE-FACTOR APP

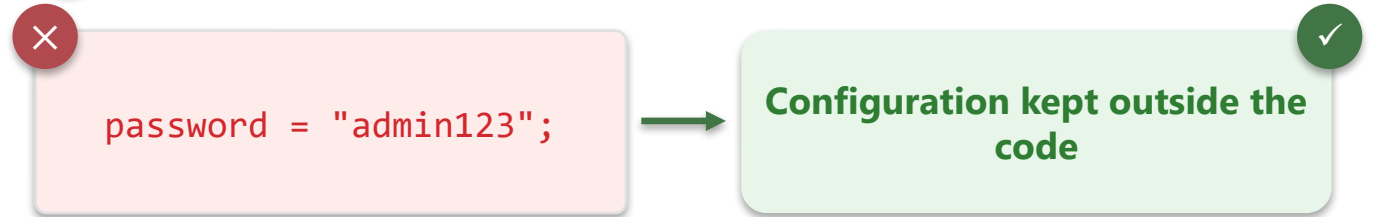
Its purpose and why it matters for cloud-native systems.

A METHODOLOGY FOR APPLICATIONS THAT ARE

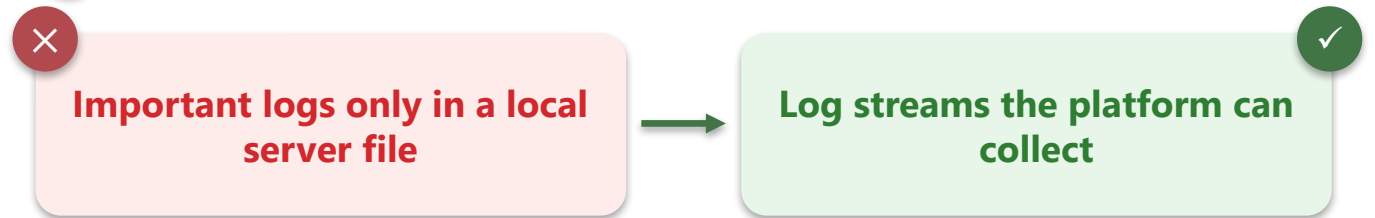


Originally designed for modern software-as-a-service applications — its principles map well to cloud-native systems and microservices.

CONFIG



LOGS



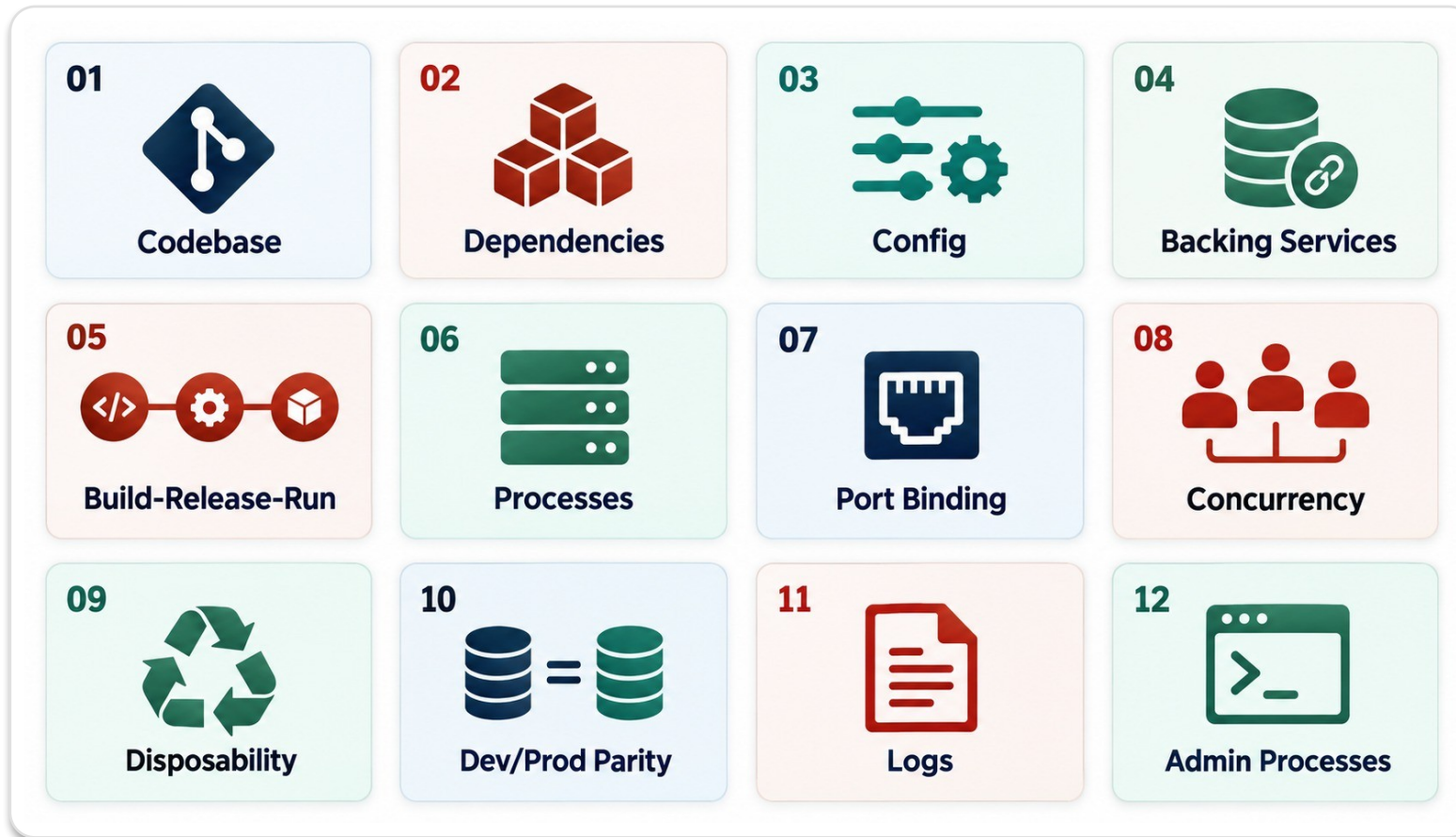
A NATURAL CONNECTION TO



KEY TAKEAWAY Twelve-Factor practices make applications portable, scalable and easy to deploy — a natural fit for OpenShift.

THE TWELVE FACTORS AT A GLANCE

All twelve principles, grouped by where they apply.



Code and build

1 Codebase · 2 Dependencies · 5 Build, release, run

Configuration

3 Config · 4 Backing services

Runtime

6 Processes · 7 Port binding · 8 Concurrency · 9 Disposability

Operations

10 Dev/prod parity · 11 Logs · 12 Admin processes

KEY TAKEAWAY This is the overview — don't spend equal time on every factor.

TWELVE-FACTOR IN A SPRING BOOT SERVICE

Config, backing services, port binding and logs — in real code.

✗ Hard-coded (breaks factors 3 and 4)

```
private String dbPassword =  
    "Prod@123";  
private String paymentUrl =  
    "https://prod-pay.shop.com";
```

Config (3)

One image runs in dev, test and prod with different values.

✓ application.yml (from env)

```
server:  
  port: ${PORT:8080}  
spring:  
  datasource:  
    url: ${DB_URL}  
    username: ${DB_USER}  
    password: ${DB_PASSWORD}  
  payment:  
    service-url: ${PAYMENT_URL}
```

Backing services (4)

The database is an attached resource found through config.

✓ Code reads config, logs to stdout

```
@Value("${payment.service-url}")  
private String paymentUrl;  
  
log.info("Order {} created for {}",  
        orderId, customerId);
```

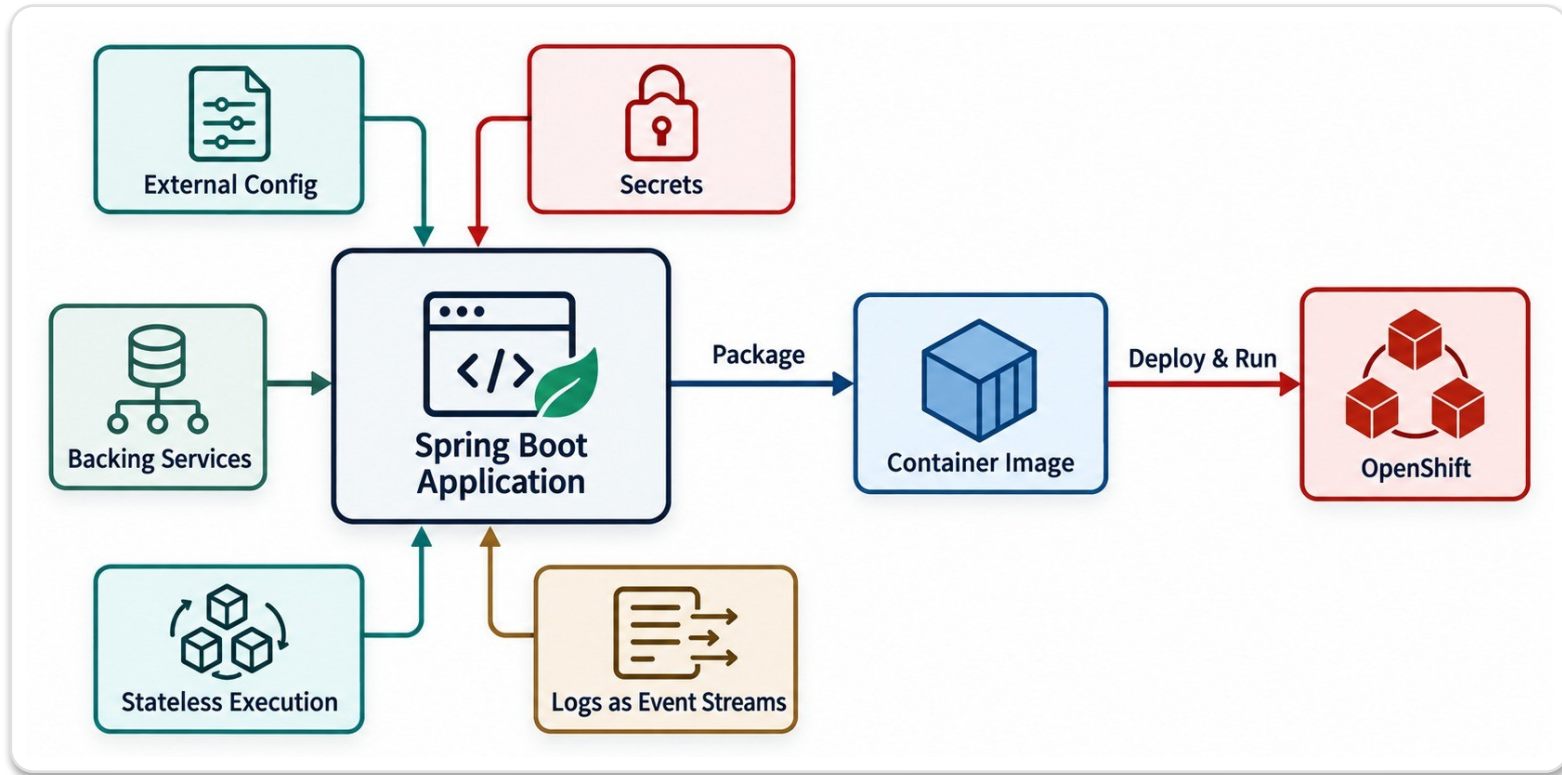
Port and logs (7, 11)

Bind to the given port; write logs to stdout for the platform.

KEY TAKEAWAY Keep configuration outside the code so one build runs everywhere.

TWELVE-FACTOR IN SPRING BOOT AND OPENSIFT

External config, stateless processes and logs — packaged and run on OpenShift.



Config

ConfigMaps and Secrets supply environment values.

Processes

Stateless pods; durable data lives in backing services.

Disposability

Pods start fast and shut down gracefully.

Logs

The platform collects the stdout stream.

KEY TAKEAWAY Twelve-Factor principles help applications deploy and run consistently across modern environments.

EXERCISE: SPOT THE TWELVE-FACTOR VIOLATIONS

Find what stops this service from running well on OpenShift.

PRACTICE EXERCISE

10 MIN

SCENARIO

A teammate wrote this part of Order Service:

```
public class OrderService {  
    String dbPassword = "Prod@123";  
    String payUrl =  
        "https://prod-pay.shop.com";  
    Map<String, Cart> carts =  
        new HashMap<>();  
  
    void save(Order o) {  
        writeFile("/var/log/orders.log", o);  
    }  
}
```

YOUR TASKS

- 1 Find each violation
- 2 Name the factor it breaks
- 3 Rewrite it the Twelve-Factor way

EXPECTED OUTCOME

- ✓ Password in code → Config (3)
- ✓ Production URL in code → Config and backing services (3, 4)
- ✓ Carts kept in memory → Processes (6)
- ✓ Local log file → Logs (11)

KEY TAKEAWAY Most Twelve-Factor problems are visible in a few lines of code.

CONWAY'S LAW AND TEAM OWNERSHIP

Team structure shapes service boundaries.

“Organizations design systems that mirror their own communication structures.” — Melvin Conway

HOW TEAMS ARE ORGANIZED

THE ARCHITECTURE YOU GET

Siloed teams: UI, backend, database



Layered services that must change together

Partly aligned teams



Inconsistent, blurry service boundaries

Teams aligned to business capabilities



Loosely coupled, independently deployable services

Stream-aligned team

Owns one service end to end — e.g. the Order team.

Platform team

Provides the paved road: pipelines, OpenShift, observability.

Enabling team

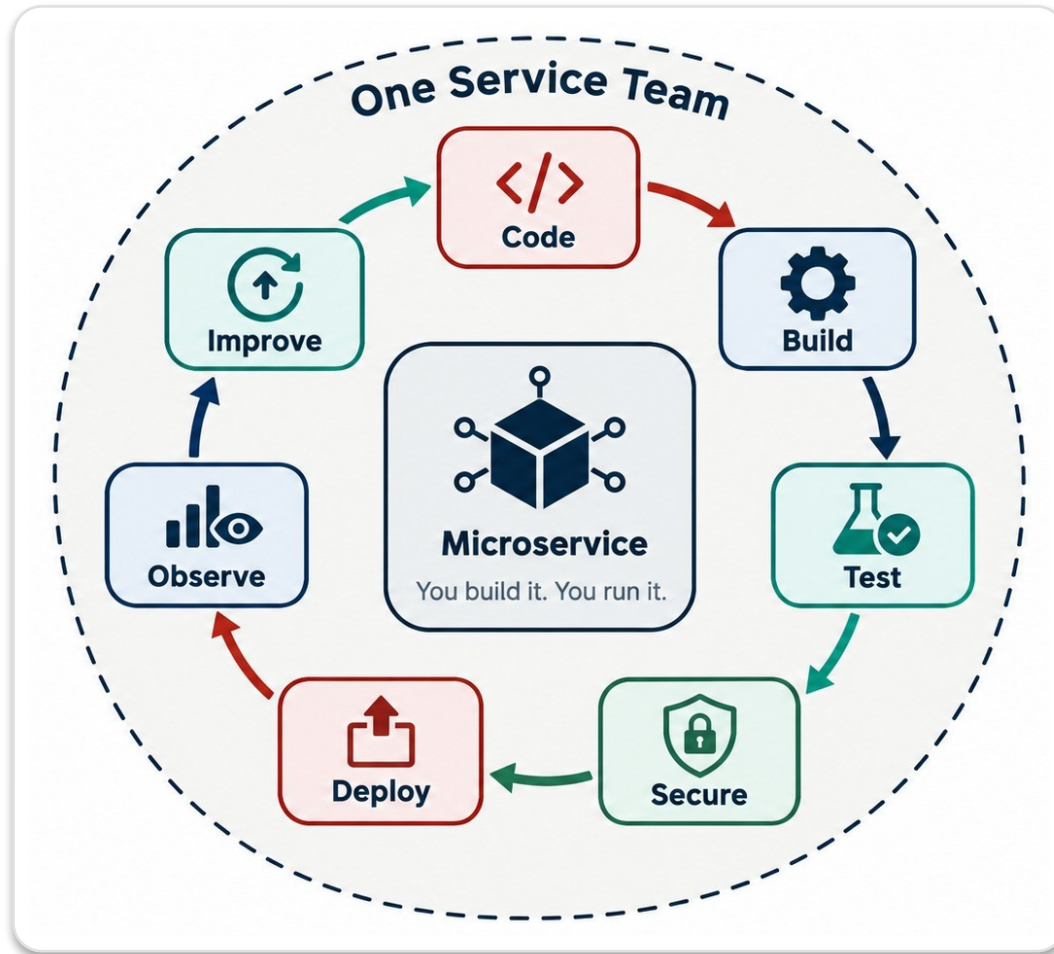
Coaches teams through new practices, then steps back.

INVERSE CONWAY Shape teams around the architecture you want.

KEY TAKEAWAY Align teams to business capabilities, and services will follow those boundaries.

HOW MICROSERVICES FIT MODERN DEVOPS TEAMS

One team owns the loop from code to improvement.



TRADITIONAL HANDOFFS

- ✗ Developer hands off to testing
- ✗ Testing hands off to release and operations

Order Service team owns

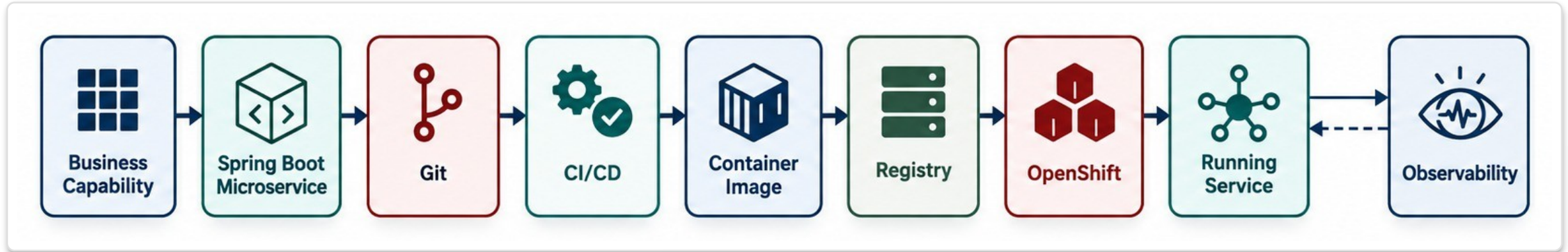
Code, tests, pipeline, deployment, security, monitoring and production support.

SHARED HELP Platform and SRE teams may provide infrastructure and tooling.

KEY TAKEAWAY Ownership models differ — platform and SRE teams may provide shared infrastructure and operations.

THE END-TO-END DELIVERY PATH

From business capability to a running, observed service on OpenShift.



Architecture

Divides the business into independently evolvable services.

Spring Boot

Builds each service.

Containers and CI/CD

Package each service and automate its delivery.

OpenShift

Schedules, scales, secures and observes the running service.

KEY TAKEAWAY Microservices, Spring Boot, containers and OpenShift are different layers that work together.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What is a microservice?

2 Why do organizations adopt microservices?

3 What is one major limitation of a monolith?

4 Are monoliths always bad? Why or why not?

5 What does independent deployment mean?

6 Why is data ownership important?

7 What is a partial failure in a distributed system?

8 What is fault isolation?

9 Why do microservices need DevOps automation?

10 Name one Twelve-Factor principle that matters for Spring Boot.

EXERCISE 1.1 — ANALYZE A MONOLITHIC BANKING APPLICATION

Module 1 checkpoint A: spot coupling and candidate boundaries.

OFFICIAL CHECKPOINT A

15 MIN

labs/day-01/exercises/exercise-1.1-analyze-monolith.md

SCENARIO

The course's banking monolith handles customer profiles, accounts, transactions, loans, fraud checks, reporting and notifications in one deployable application.

Apply what you learned on the order platform to this system.

YOUR TASKS

- 1 List the major business capabilities in the monolith
- 2 Mark tightly coupled areas and modules that change often or carry high traffic
- 3 Mark where failure should be isolated so money movement is not taken down by reporting
- 4 Name the first extraction candidate and say why

EXPECTED OUTCOME

- ✓ A short list of coupling problems
- ✓ One justified first extract: Account Service
- ✓ Lab 1 builds Account Service after Modules 1–3
- ✓ Design only — no code in this checkpoint

KEY TAKEAWAY A short list of coupling problems and one justified first extract: Account Service.

MODULE 1 SUMMARY AND WHAT'S NEXT

From why microservices exist to how they run.

YOU CAN NOW EXPLAIN

- ✓ Why monoliths struggle to change and scale
- ✓ What makes a service a microservice
- ✓ How services own data and communicate
- ✓ What independence costs — and how to design for failure
- ✓ When a monolith or modular monolith is the better choice
- ✓ Twelve-Factor habits and DevOps ownership

NEXT: MODULE 2 — MICROSERVICES DESIGN

- 1 Where exactly should service boundaries go?
- 2 How should service APIs be designed?
- 3 How should services communicate — and where does state live?

BRING ALONG The order platform, the five principles and your Exercise 1.1 notes.

KEY TAKEAWAY Module 2 turns these principles into concrete service boundaries, APIs and communication choices.